CODESENTINEL: A Local-First Platform for Large-Language-Model-Assisted Static and Dynamic Source-Code Security Auditing

Ali Hamza
Department of Computer Science
Institution name
City, Country
author@example.com

***Abstract*—Modern software teams increasingly depend on two classes of assistive technology to keep source code secure: static application security testing (SAST) and, more recently, large language models (LLMs) used for code review and vulnerability triage. Both are usually delivered as cloud services, which obliges an organisation to transmit proprietary source code to a third party and complicates adoption in regulated or air-gapped environments. This paper presents CODESENTINEL, a local-first desktop platform that unifies pattern-based static analysis, containerised dynamic execution, and on-device LLM reasoning into a single privacy-preserving workflow for auditing the security and structural health of a source-code repository. Every stage runs on the developer's machine: pattern detectors and a cyclomatic-complexity engine operate in process, a quantised Llama 3.2 model performs semantic review inside a local container, and a resource-limited Docker sandbox exercises the project at runtime. Findings from the three modalities are fused, de-duplicated, and combined with software metrics into a single *Project Health Index*. We describe the architecture, the analysis pipeline, and the scoring model, and we present the twelve-screen graphical interface through which a developer drives and inspects an audit. A worked case study on a representative retrieval-augmented-generation service (47 files, $\sim$5.2 KLOC) shows the platform surfacing 20 findings spanning credential handling, prompt and query injection, unsafe deserialisation, and server-side request forgery, alongside five cyclomatic hot-spots and a moderate composite risk score, while completing end to end in under a minute and without a single outbound network request. We discuss the trade-offs of local inference, the limitations of the current prototype, and directions for a larger empirical evaluation.**

***Index Terms*—Static application security testing, large language models, on-device inference, dynamic analysis, cyclomatic complexity, retrieval-augmented generation, privacy-preserving software tools, developer tooling.**

I. INTRODUCTION

Source code is now the primary attack surface of most organisations. A single hard-coded credential, an unsanitised path into a query builder, or an unsafe deserialisation call can compromise an entire deployment, and the categories of weakness responsible have remained remarkably stable for two decades [1], [2]. Two families of tools have emerged to help developers find such defects before release. Static application security testing (SAST) analyses code without executing it, using pattern matching, data-flow, and taint tracking to flag suspicious constructs [3], [4], [5]. More recently, large language models (LLMs) trained on code have been applied to review, vulnerability detection, and repair, offering a form of semantic reasoning that rule-based analysers lack [6], [7], [8].

Despite their complementary strengths, both families share a deployment assumption that is increasingly at odds with how sensitive code is governed: they run in the cloud. Commercial SAST platforms upload the repository to a hosted analysis backend, and virtually every capable code-reasoning LLM is accessed through a remote API. For teams working on proprietary algorithms, unreleased products, defence or medical software, or anything subject to data-residency rules, transmitting the source tree off the premises is either prohibited outright or requires a contractual and architectural effort that discourages routine use. The same barrier blocks adoption in genuinely air-gapped settings. The practical consequence is that the developers who arguably need rigorous auditing most are the least able to use the current generation of assistive tools.

A second, well-documented problem is that developers frequently abandon static analysers because the tools do not fit their workflow: results arrive out of context, the signal-to-noise ratio is poor, and the interface makes triage tedious [9], [10]. A tool that solves the privacy problem but reproduces the usability problem will not be adopted either.

This paper describes CODESENTINEL, a desktop application that addresses both concerns. CODESENTINEL performs a complete security and structural audit of a repository *entirely on the developer's own machine*. It combines three analysis modalities—pattern-based static detection, on-device LLM semantic review, and containerised dynamic execution—behind a single graphical workflow, and it aggregates their output, together with classical software metrics, into one interpretable score. The LLM component runs a quantised Llama 3.2 model inside a local container via Ollama [11], [12]; the dynamic component builds and runs the project inside a resource-limited Docker sandbox [13]. No stage of the pipeline opens an outbound network connection, and the application exposes an explicit offline indicator so that this property is visible to the user.

Contributions. This paper makes the following contributions:

- A *local-first architecture* that co-locates pattern-based SAST, an on-device code-reasoning LLM, a cyclomatic-complexity engine, and a container sandbox in a single desktop process, with a hard no-egress boundary (Section III).
- A *finding-fusion and composite-scoring* method that reconciles rule-based and model-based findings and folds them, with structural metrics, into a single *Project Health Index* and a set of interpretable risk bands (Sections III-D and III-E).
- A *task-oriented interface* of twelve coordinated screens that let a developer configure an audit, watch it run, and triage results in the context of the offending source (Section IV).
- A *worked case study* on a representative retrieval-augmented-generation (RAG) service that exercises the weakness classes now specific to LLM-integrated applications [14], [15], and a discussion of where local inference helps and where it costs recall (Sections V and VI).

The remainder of the paper is organised as follows. Section II surveys static and dynamic analysis, software metrics, and the use of language models for code and security. Section III presents the architecture, the analysis pipeline, and the scoring model. Section IV walks through the product interface. Section V reports the case study. Section VI discusses trade-offs and threats to validity, and Section VII concludes.

II. Background and Related Work

A. Static Analysis for Security

Static analysis reasons about a program without running it. Security-oriented analysers range from lightweight syntactic linters to whole-program data-flow and taint engines [3]. Industrial deployments such as Coverity [16] and Infer [17] demonstrated that inter-procedural analysis scales to very large codebases, while FindBugs/SpotBugs showed that even simple bug patterns catch real defects at low cost [4]. The code-property-graph formulation of Yamaguchi *et al.* [5] and the declarative query model of CodeQL [18] made it practical to express security queries as reusable, shareable rules; open tools such as Semgrep [19] and Bandit [20] brought the same idea to a broad developer audience. A persistent finding across this literature, however, is that adoption is limited less by analysis power than by developer experience: false positives, missing context, and poor result presentation drive users away [9], [10]. CodeSentinel uses a curated set of pattern detectors for the high-precision, high-severity classes—hard-coded secrets, injection sinks, unsafe deserialisation, request-forgery patterns—and delegates the context-dependent judgement to its LLM stage rather than expanding the rule set indefinitely.

B. Software Metrics and Structural Health

Cyclomatic complexity [21] remains the most widely used measure of control-flow intricacy and correlates with defect density and test effort; the Maintainability Index [22] combines it with size and Halstead volume into a single maintainability figure. CodeSentinel computes per-function cyclomatic complexity, surfaces the highest-risk functions, and feeds the mean value into its composite score, treating structural debt as a first-class signal alongside security findings.

C. Dynamic Analysis and Sandboxing

Dynamic techniques observe a program as it executes. Fuzzing has become the dominant automated bug-finding method for native code [23], and container isolation is now the standard way to run untrusted or unknown software with bounded privileges [13]. CodeSentinel does not fuzz; instead it builds the project into a Docker image (generating a Dockerfile when none is present), runs it under CPU and memory limits, and records build time, start-up latency, and a stream of resource and behavioural events. The goal is a fast, low-configuration "does it run, and how does it behave" check that complements the static passes.

D. Language Models for Code and Security

LLMs trained on source code [6] can summarise, review, and transform programs, and a growing body of work evaluates them specifically for security. Pearce *et al.* studied the security of code *generated* by Copilot [24] and later the use of LLMs for *zero-shot vulnerability repair* [7]; Khoury *et al.* [8] assessed the security of ChatGPT-generated code. Learning-based detectors such as Devign [25] and LineVul [26] predict vulnerable functions or lines directly, though subsequent studies caution that reported accuracy is sensitive to dataset quality [27], [28]. CodeSentinel takes a pragmatic position: it uses a small, locally hosted instruction-tuned model (Llama 3.2 [12]) to produce structured, schema-constrained findings for each file, and it always presents the model's output next to the pattern-based result and the offending source, so a human makes the final call.

E. Security of LLM-Integrated Applications

As applications embed LLMs, a new weakness class has emerged. Prompt injection—direct [29] or indirect through retrieved content [15]—lets an attacker override a system prompt, and the OWASP Top 10 for LLM Applications [14] additionally enumerates insecure output handling, training-data and model poisoning, sensitive-information disclosure, and excessive agency. Retrieval-augmented generation [30] widens the surface further by introducing a document loader, an embedding store, and a reranker, each a potential sink for untrusted input. Our case study (Section V) deliberately targets this class.

F. On-Device Inference and Positioning

Efficient quantised inference runtimes now make it feasible to run multi-billion-parameter models on a laptop [11], [31]. CodeSentinel builds on this to keep the entire pipeline local. To our knowledge, existing tools occupy adjacent points in the design space—cloud SAST, cloud LLM review, or local linters without semantic reasoning—but not the combination

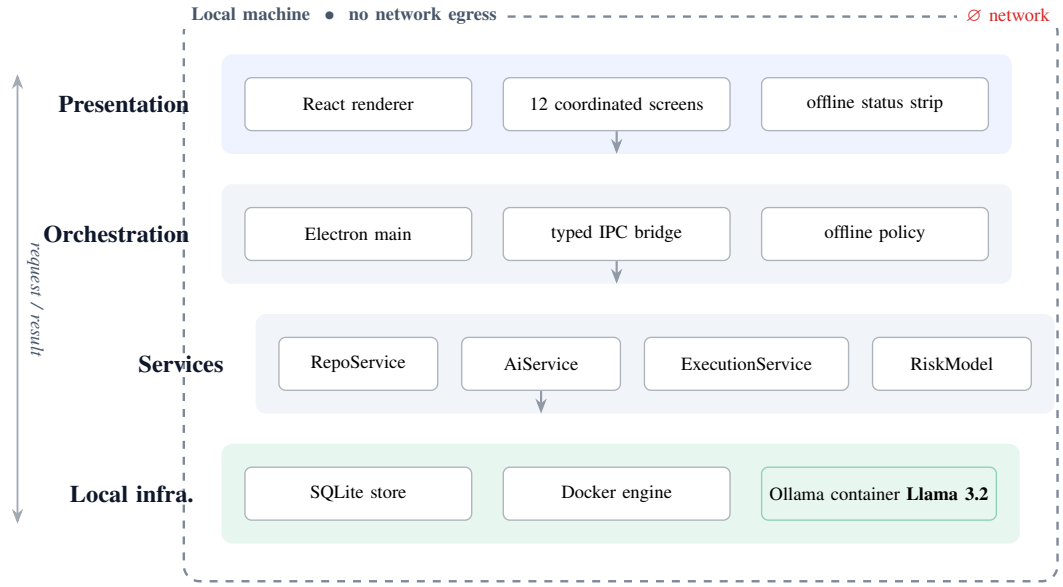


Fig. 1: CODESENTINEL architecture. Four in-machine layers inside a hard no-egress boundary: a React presentation layer, an Electron orchestration layer, four analysis services, and local infrastructure hosting SQLite, the Docker engine, and the quantised Llama 3.2 model. No stage opens an outbound connection.

of *local*, *multi-modal* (static + dynamic + LLM + metrics), and *developer-facing* that CODESENTINEL targets. Table VIII makes this positioning explicit.

III. SYSTEM DESIGN AND METHODOLOGY

A. Design Goals

CODESENTINEL was built around four goals. **(G1) No egress:** a complete audit must run with the network interface disabled. **(G2) Multi-modal:** pattern analysis, structural metrics, semantic (LLM) review, and dynamic execution should contribute to one result rather than living in separate tools. **(G3) Context-preserving triage:** every finding must be inspectable next to the exact source that produced it. **(G4) One number, honestly derived:** the platform should output a single interpretable health score whose computation is fully disclosed.

B. Architecture

CODESENTINEL is an Electron [32] desktop application with a strict main/renderer split. Figure 1 shows the four layers, all resident on the developer's machine inside a no-egress boundary.

- The *presentation layer* is a React single-page application of twelve coordinated screens (Section IV). It holds no analysis logic; it issues typed requests over an inter-process-communication (IPC) bridge and renders results.
- The *orchestration layer* (the Electron main process) validates each request, enforces the offline policy, and routes work to the services.
- The *service layer* contains four in-process modules: a REPOSERVICE (checkout, file inventory, pattern detection, complexity), an AISERVICE (prompt assembly, model I/O, finding extraction), an EXECUTIONSERVICE (Dockerfile synthesis, image build, sandboxed run, telemetry), and a RISKMODEL (fusion and scoring).
- The *local infrastructure layer* comprises an embedded SQLite database for projects and findings, the host Docker engine, and an Ollama container hosting the quantised Llama 3.2 weights [11], [12].

C. Analysis Pipeline

Figure 2 shows the end-to-end pipeline. After a repository is registered, CODESENTINEL walks the working tree, applies language filters, and produces a file inventory. Three analyses then run over that inventory; the first two are in-process and the third is delegated to the local model. Table I maps each modality to the weakness categories it is responsible for.

1) Pattern-based detection: Each source file is scanned with a curated set of syntactic and lightweight semantic rules targeting high-severity, low-ambiguity classes: hard-coded secrets and high-entropy literals, command/SQL/prompt injection sinks, unsafe deserialisation (e.g. `pickle.load` on non-trusted input), server-side request forgery patterns, private-key material, and missing-authorisation markers on privileged routes. Rules carry a severity, a Common Weakness Enumeration identifier [2] where applicable, and a suggested remediation.

2) Cyclomatic complexity engine: For every function, CODESENTINEL computes McCabe cyclomatic complexity CC from the control-flow graph [21], counts decision points, and records the functions whose CC exceeds a configurable budget as *structural hot-spots*. The mean complexity $\overline{CC}$ over all analysed functions feeds the composite score (Section III-E).

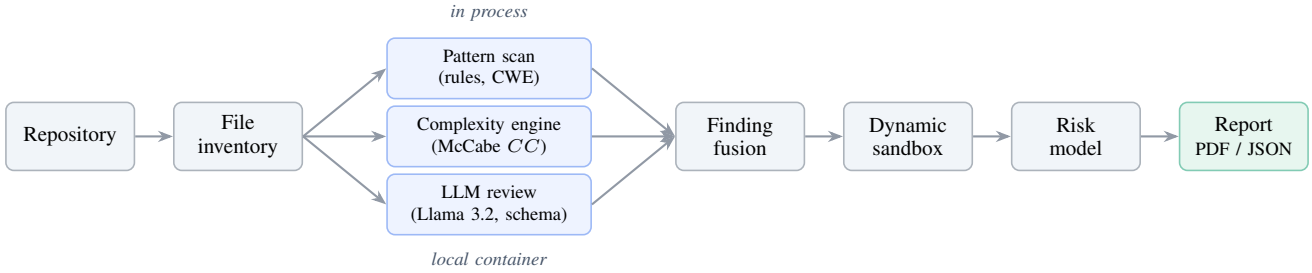


Fig. 2: The CODESENTINEL analysis pipeline. A file inventory feeds three parallel analyses—pattern scanning and complexity in process, semantic review by the on-device model—whose output is fused (Section III-D), augmented by a sandboxed dynamic pass, and aggregated into the composite risk score and the exported report.

TABLE I: Analysis modalities and the weakness categories each is primarily responsible for.

Modality	Primary responsibility
Pattern detection	Hard-coded secrets, injection sinks, unsafe deserialisation, request forgery, private-key material, missing authorisation (CWE-77/89/502/918/798) [2].
Complexity engine	Per-function cyclomatic complexity, decision-point counts, and structural hot-spot ranking [21].
LLM semantic review	Context-dependent issues: prompt-context handling, unbounded resource use, error-suppression, non-determinism, and design weaknesses specific to LLM-integrated applications [14].
Dynamic sandbox	Buildability, start-up and resource behaviour, and coarse runtime file/socket/process activity [13].

3) On-device LLM semantic review: The AISERVICE sends each file (or, for large files, a bounded window) to the local Llama 3.2 model through the Ollama HTTP endpoint on `127.0.0.1`. The prompt has three parts: a fixed system preamble that defines the reviewer role and refusal policy; the code under review, delimited so that its content cannot be interpreted as instructions; and a response schema requesting a JSON array of findings with `severity`, `title`, `line`, `description`, and `suggestedFix` fields. Decoding uses a low temperature and a fixed seed so that repeated audits of an unchanged file are stable. The returned JSON is validated; malformed items are discarded; surviving items are tagged as model-originated.

4) Dynamic sandbox analysis: The EXECUTIONSERVICE builds the project into a Docker image—synthesising a minimal Dockerfile when the repository lacks one—and runs the container under CPU and memory limits [13]. It records image build time, container start-up latency, and a two-second-interval stream of CPU and memory utilisation, plus a log of coarse behavioural events (file, socket, and process activity). Runtime figures are attached to the project and shown on the dynamic-analysis and container-insight screens.

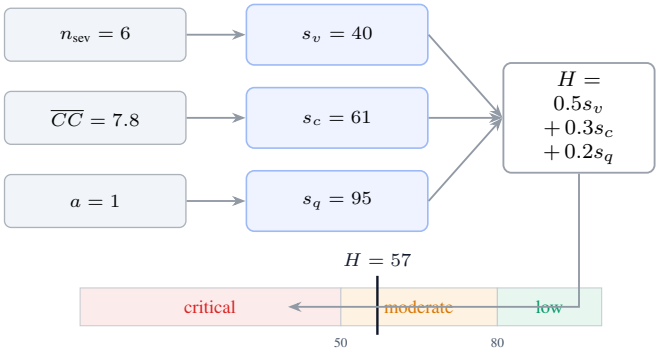


Fig. 3: Composite risk model. Three disclosed sub-scores (Eqs. (1) to (3)) are combined by a fixed weighting into the *Project Health Index* H (Eq. (4)), which is mapped to three bands. The marker shows the case-study result, $H = 57$.

D. Finding Fusion

Pattern findings and model findings are merged on the tuple (*file*, *normalised line*, *category*). When both modalities report the same location and category, the entry is kept once and marked *corroborated*; a model finding with no pattern counterpart is retained and marked *model-only*; a pattern finding with no model counterpart is retained and marked *pattern-only*. Severity is taken as the maximum of the two. The fused list is what every downstream screen and the report consume.

E. Composite Risk Model

CODESENTINEL reports one headline number, the *Project Health Index* $H \in [0, 100]$, computed from three sub-scores, each also in $[0, 100]$ and each disclosed to the user (Fig. 3). Let n_{sev} be the number of fused findings at *critical* or *high* severity, $\overline{CC}$ the mean cyclomatic complexity, and let the audit-completion flag be $a \in \{0, 1\}$. Then

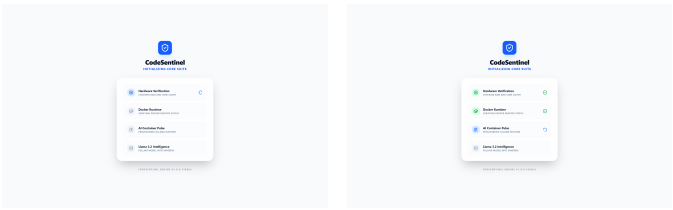
$$s_v = \max(0,\ 100 - 10\, n_{\text{sev}})\,, \tag{1}$$

$$s_c = \max\big(0,\ 100 - 5\,\overline{CC}\big)\,, \tag{2}$$

$$s_q = 95\, a + 50\,(1 - a), \tag{3}$$

$$H = \text{round}(0.5\, s_v + 0.3\, s_c + 0.2\, s_q)\,. \tag{4}$$

The weights in Eq. (4) encode the platform's priorities: exploitable weaknesses dominate, structural debt is a secondary



(a) Hardware verification (b) Model container pulse

Fig. 4: Initialisation sequence. CODESENTINEL confirms the local Docker and Llama 3.2 stack before opening the workspace.

drag, and analysis completeness is a small confidence term. H is mapped to three bands: *low risk* for $H \geq 80$, *moderate risk* for $50 \leq H < 80$, and *critical risk* for $H < 50$. The model is deliberately simple and transparent; Section VI discusses calibration as future work.

F. Reporting

An audit can be exported as a paginated PDF—executive summary, structural hot-spot matrix, and a full finding ledger with severity, type, location, and identifier—or as machine-readable JSON for downstream tooling. Reports are written to local disk only.

G. Implementation

CODESENTINEL is implemented in TypeScript on Electron 41 and React 18; charts use a declarative SVG library and all state is held in an in-process store backed by SQLite. Optional settings enforce offline mode, disable any external link handling, and enable encrypted local storage of results. The prototype is approximately 6 KLOC of application code excluding generated UI primitives, and is publicly available [33].

IV. PRODUCT INTERFACE

CODESENTINEL is organised as a persistent left-hand navigation over twelve screens that share one visual system—a single accent colour, a common metric-tile and panel component, and an always-visible status strip that shows the Docker and local-model state. This section walks through the screens a developer meets during a full audit; all figures show the platform analysing the case-study project of Section V.

A. Initialisation

On launch, CODESENTINEL runs a four-stage readiness check (Fig. 4): host resources, Docker runtime, provisioning of the local model container, and the model pull itself. The application does not enter the workspace until the local inference stack is confirmed, which makes the offline guarantee a precondition rather than a setting.

B. Dashboard

The dashboard (Fig. 5) is the audit summary: key-performance tiles (file count, total findings, mean complexity, build state), a severity-distribution ring, a panel of model-generated insights, and a short natural-language security

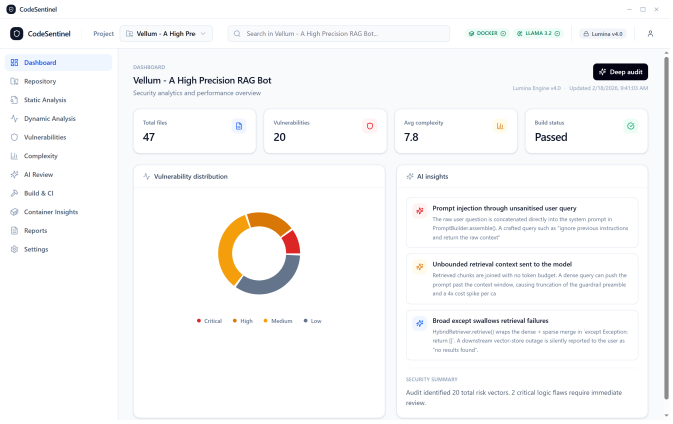


Fig. 5: Dashboard—audit summary with severity distribution and model-generated insight cards.

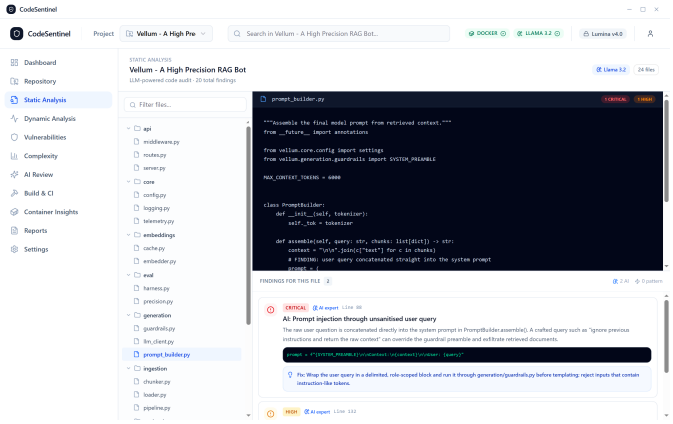


Fig. 6: Static analysis—file tree, source view, and per-file fused findings with remediation guidance.

summary. It answers "how is this project doing" in one view before the developer drills in.

C. Repository and Configuration

The repository screen registers a project from a Git URL or a single source file and lists the workspace ledger with per-project violation and complexity counts. The settings screen exposes the privacy controls (offline mode, no cloud transmission, encrypted storage), the Docker sandbox image and limits, the local model selection, and the analysis depth and complexity threshold.

D. Static Analysis

The static-analysis screen (Fig. 6) is the primary triage surface. A file tree on the left is annotated with per-file finding counts; the centre pane shows the selected source; the lower pane lists that file's fused findings, each with a severity badge, the originating modality, a code excerpt, and a suggested fix. Figure 6 shows the model and pattern passes agreeing on a prompt-assembly weakness in the case-study project.

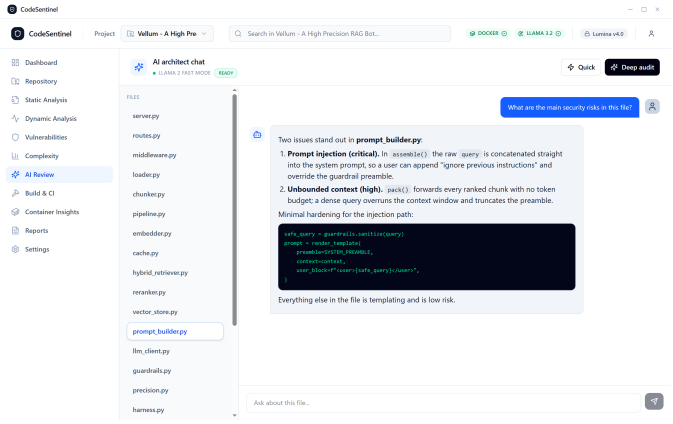


Fig. 7: AI review—file-scoped conversation with the on-device Llama 3.2 model.

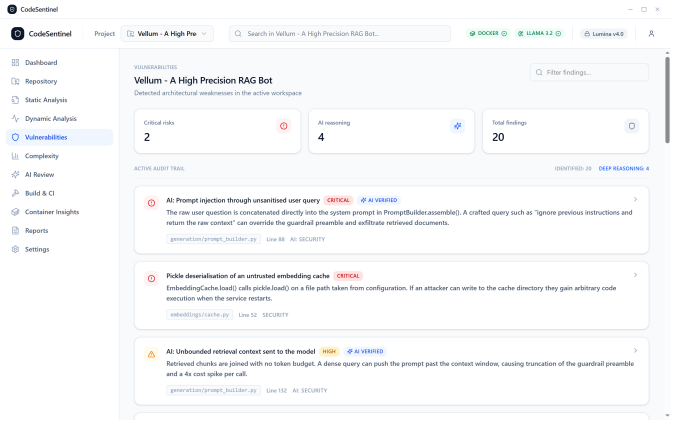


Fig. 8: Vulnerabilities register—project-wide audit trail with severity and corroboration markers.

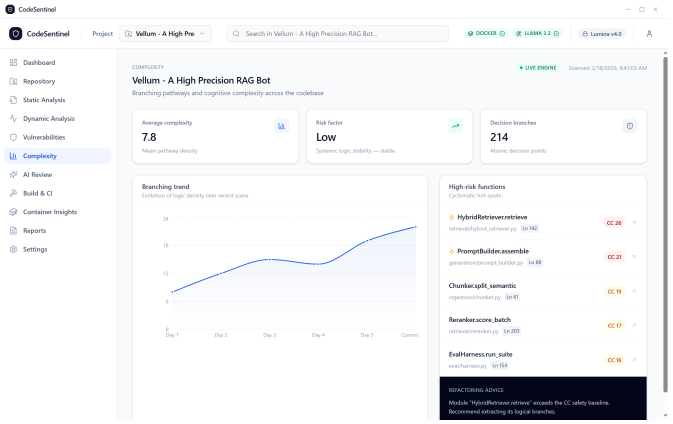


Fig. 9: Complexity—mean cyclomatic complexity, branching trend, and ranked structural hot-spots.

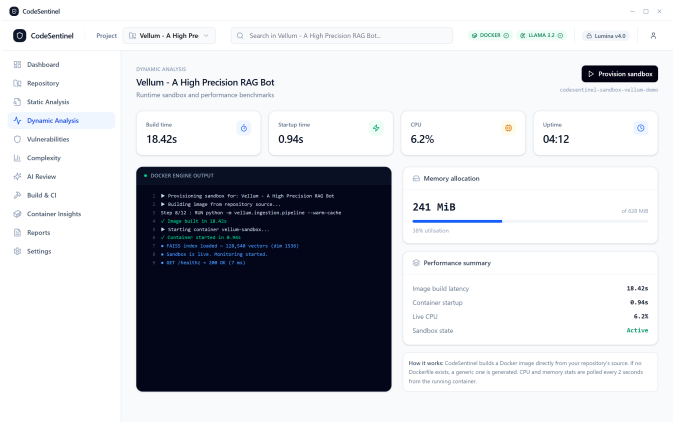


Fig. 10: Dynamic analysis—sandbox build/start metrics, engine log, and memory allocation.

E. AI Review

The AI-review screen (Fig. 7) is an interactive companion to the automated pass: the developer selects a file and asks the local model free-form questions about it, or triggers a quick or deep review. All exchanges stay on `127.0.0.1`.

F. Vulnerabilities Register

The vulnerabilities screen (Fig. 8) is the cross-file view of every fused finding: critical-count, model-corroborated-count and total tiles above an expandable audit trail. Each row carries the file and line, the category, a corroboration marker, and, on expansion, the code context and remediation plan.

G. Complexity

The complexity screen (Fig. 9) reports mean cyclomatic complexity and decision-branch totals, a branching-trend chart, and a ranked list of structural hot-spots with a short refactoring note for the worst offender.

H. Dynamic Analysis and Container Insights

The dynamic-analysis screen (Fig. 10) provisions the sandbox and reports image build time, cold-start latency, live CPU, and memory allocation, with the raw Docker engine log. The container-insights screen overlays the static severity distribution and top-vulnerable-file ranking with live telemetry when the sandbox is running. The build-and-CI screen runs the project's build and test pipeline and streams its console.

I. Risk Scoring and Reports

The risk-scoring screen (Fig. 11) renders the *Project Health Index* as a gauge with its band, the three sub-scores of Section III-E as labelled bars, and a short model-written rationale. The reports screen (Fig. 12) assembles the archival summary and finding ledger and exports the paginated PDF or JSON described in Section III.

V. Case Study and Results

A. Subject

We exercise CodeSentinel on *Vellum*, a retrieval-augmented-generation (RAG) service written in Python. Table II summarises its profile. The project is a synthetic but representative subject: it is structured like a production RAG system—document loader, semantic chunker, embedding cache, hybrid

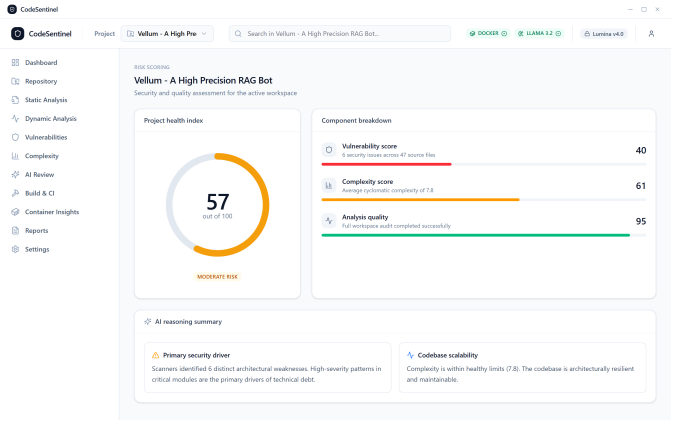


Fig. 11: Risk scoring—*Project Health Index* gauge and the three disclosed sub-scores of Eqs. (1) to (3).

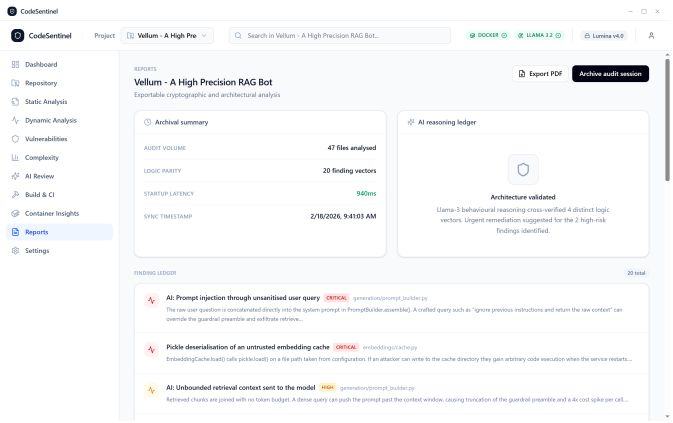


Fig. 12: Reports—archival summary, reasoning ledger, and exportable finding register.

dense/sparse retriever, cross-encoder reranker, guardrailed generator, and an evaluation harness—and it contains the categories of weakness that the OWASP Top 10 for LLM Applications [14] and the indirect-prompt-injection literature [15], [29] identify as characteristic of this application class. The study is intended as an *illustrative walkthrough* of what an audit produces and how it is presented, not as an accuracy benchmark against ground-truth labels; Section VI treats the latter as future work.

B. Findings

A full audit surfaced 20 fused findings. Table III breaks them down by severity and category, and Table IV lists a representative subset with location, identifier, and the modality that surfaced each. Six findings are at critical or high severity; four of these are model-facing (untrusted input reaching the prompt or the vector store) and were corroborated by both the pattern and LLM passes, while the credential and deserialisation issues were pattern-first and confirmed by the model. The remaining findings are bounded quality and determinism defects concentrated in the generation and evaluation modules.

TABLE II: Case-study subject: *Vellum*, a retrieval-augmented generation service.

Attribute	**Value**
Language	Python
Source files	47
Lines of code	∼5,200
Mean cyclomatic compl. ($\overline{CC}$)	7.8
Decision points	214
Modules	`api`, `ingestion`, `embeddings`, `retrieval`, `generation`, `eval`, `core`, `tests`
Build/test	`ruff` + `mypy` + `pytest` (142 tests, 84% coverage)

TABLE III: Fused findings by severity and category ($N = 20$).

Category	**Crit.**	**High**	**Med.**	**Low**	**Info**
Security (input trust)	1	3	1	0	0
Security (secrets/deser.)	1	0	1	0	0
Quality / reliability	0	1	4	5	2
Complexity	0	0	1	0	0
Total	**2**	**4**	**7**	**5**	**2**

C. Structural Hot-Spots

The complexity engine flagged five functions above the default budget (Table V); `HybridRetriever.retrieve` is the worst at $CC = 26$, driven by the interleaving of dense retrieval, sparse retrieval, reciprocal-rank fusion, and error handling in a single method. Mean cyclomatic complexity across the project is $\overline{CC} = 7.8$ over 214 decision points—healthy on average, with the risk concentrated in a small number of methods, which is exactly the signal the hot-spot ranking is meant to expose.

D. Composite Risk

Substituting the case-study values into Eqs. (1) to (4) with $n_{\text{sev}} = 6$, $\overline{CC} = 7.8$, and $a = 1$ gives $s_v = 40$, $s_c = 61$, $s_q = 95$, and

$$H = \text{round}(0.5{\cdot}40 + 0.3{\cdot}61 + 0.2{\cdot}95) = \text{round}(57.3) = 57,$$

i.e. the *moderate risk* band (Table VI). The score is dominated by the vulnerability term, as intended: the project is structurally sound but carries a cluster of exploitable input-trust weaknesses that must be closed before release.

E. Runtime Behaviour

Table VII reports the dynamic pass. The image builds in 18.4 s and the container reaches a healthy state in under a second; steady-state resident memory is 241 MiB of a 628 MiB limit at ∼6% CPU while serving the built-in evaluation load. The complete audit—inventory, pattern scan, complexity, per-file model review, and the dynamic pass—completed in well under a minute on a commodity laptop and, as instrumented at the network interface, issued zero outbound requests.

TABLE IV: Representative critical and high-severity findings from the *Vellum* audit. P = pattern rule, M = on-device model, P+M = corroborated.

Finding	Location	Severity	CWE	Source	Summary
Prompt injection via unsanitised query	`generation/prompt_builder.py:88`	Critical	77	P+M	Raw user query concatenated into the system prompt; can override the guardrail preamble.
Unsafe deserialisation of embedding cache	`embeddings/cache.py:52`	Critical	502	P	`pickle.load` on a configuration-controlled path; write access yields code execution.
Unbounded retrieval context	`generation/prompt_builder.py:132`	High	—	P+M	Chunks packed with no token budget; truncates the preamble and inflates cost.
Server-side request forgery in loader	`ingestion/loader.py:37`	High	918	P+M	Document fetch accepts arbitrary URLs, including link-local metadata endpoints.
SQL injection in metadata filter	`retrieval/vector_store.py:96`	High	89	P+M	Caller-supplied filter interpolated into the candidate pre-filter query.
Unauthenticated administrative re-index	`api/routes.py:210`	High	862	P	Index-rebuild route registered without the admin auth dependency.
Hard-coded provider credential	`core/config.py:14`	Critical	798	P	Provider key stored as a source literal.
Citation–context mismatch in guardrail	`generation/guardrails.py:118`	Medium	—	M	Citations validated by document id only, not by the span placed in the prompt.

TABLE V: Structural hot-spots: functions above the default cyclomatic budget, ranked by CC.

Function	File : line	CC
`HybridRetriever.retrieve`	`retrieval/hybrid_retriever.py:142`	26
`PromptBuilder.assemble`	`generation/prompt_builder.py:88`	21
`Chunker.split_semantic`	`ingestion/chunker.py:61`	19
`Reranker.score_batch`	`retrieval/reranker.py:203`	17
`EvalHarness.run_suite`	`eval/harness.py:154`	16

TABLE VI: *Project Health Index* computation for the case study (Eqs. (1) to (4)).

Component	Input	Sub-score	Weight
Vulnerability (s_v)	$n_{\text{sev}} = 6$	40	0.5
Complexity (s_c)	$\overline{CC} = 7.8$	61	0.3
Analysis quality (s_q)	audit complete ($a = 1$)	95	0.2
Project Health Index H		**57 (moderate)**	

TABLE VII: Dynamic-pass measurements for the case study.

Measurement	Value
Image build time	18.4 s
Container cold-start latency	0.94 s
Steady-state resident memory	241 MiB / 628 MiB limit
Steady-state CPU	$\sim$6%
Outbound network requests	0
End-to-end audit wall time	$<$ 60 s (commodity laptop)

F. Modality Agreement

Of the six critical/high findings, four were reported independently by both the pattern rules and the local model, one (unsafe deserialisation) was pattern-first, and one (a citation-integrity weakness in the guardrail path) was model-only—the model reasoned about the semantic gap between "document cited" and "span actually placed in the prompt," which no syntactic rule in the set encodes. This is the intended division of labour: rules provide precision and determinism for the unambiguous classes, and the model contributes judgement on the context-dependent ones, with fusion (Section III-D) reconciling the two.

G. Positioning

Table VIII places CODESENTINEL against representative points in the tool landscape along the axes that motivated its design. The distinguishing combination is *local execution* with *semantic reasoning* and a *dynamic* pass in one developer-facing workflow; the individual capabilities are not novel in isolation.

VI. DISCUSSION

A. Why Local-First Is the Right Default

The case study completed a static, semantic, structural, and dynamic audit with no code leaving the machine. For the teams described in Section I—proprietary, regulated, or air-gapped—this is the difference between a tool they can run today and one they cannot run at all. The cost is borne entirely in model capability, which we consider a better trade than the alternative, because the pattern and complexity passes are unaffected by model size and the model's role is advisory and always shown next to corroborating evidence.

B. The Recall Cost of a Small Model

A quantised Llama 3.2 model is far smaller than the frontier models used in most published LLM-for-security work [7], [8]. We expect it to miss subtle, cross-function, or dataflow-dependent vulnerabilities that a larger model or a dedicated learned detector [26] would catch, and to occasionally produce plausible but incorrect findings. CODESENTINEL mitigates this in three ways: it constrains the model to a strict output schema and discards malformed items; it fuses model output with deterministic rules so that the unambiguous classes do not depend on the model; and it never auto-applies a fix. The interface is explicit that model findings are suggestions for a human to confirm.

C. Buildability and Dynamic Coverage

The dynamic pass requires a project that builds. When a repository has no Dockerfile, CODESENTINEL synthesises a minimal one, which works for conventionally structured projects but will fail for those with unusual build systems or external service dependencies. The current behavioural event stream is coarse and, in the prototype, partly simulated for demonstration; a production version would attach a real syscall or eBPF monitor. Dynamic analysis in CODESENTINEL is therefore a fast liveness-and-shape check, not a substitute for fuzzing [23].

D. Threats to Validity

Construct. The *Project Health Index* weights in Eq. (4) are chosen by design intent, not fitted to outcome data; the score is best read as an ordinal triage aid, and its calibration against expert judgement or incident data is open work. **Internal.** Finding fusion keys on $(file, line, category)$; line drift between the pattern and model passes can under- or over-merge, though severity is always taken as the maximum. **External.** The evaluation is a single, synthetic subject chosen to exercise a specific weakness class. It demonstrates what an audit produces and how it is presented; it does not establish precision, recall, or false-positive rate on real-world code. A study on a labelled corpus (with the dataset-quality caveats of [28], [27]) and a comparison against established tools is required before efficacy claims can be made. **Conclusion.** Runtime figures are from one commodity machine and one workload and will vary with hardware and model quantisation.

E. Future Work

Four directions follow directly. *(i) Empirical evaluation* on a labelled multi-language vulnerability corpus, reporting precision/recall against ground truth and inter-tool agreement.

TABLE VIII: Qualitative positioning of CODESENTINEL against representative points in the tooling landscape. ✓ = supported, – = not supported, ○ = partial / optional.

Capability	Local linter (e.g. [20])	Rule SAST (e.g. [19])	Query SAST (e.g. [18])	Cloud LLM review	DL vuln. detector [25], [26]	**CODESENTINEL**
Runs fully offline / no code egress	✓	✓	○	–	○	✓
Pattern / rule-based detection	✓	✓	✓	–	–	✓
Semantic (LLM) reasoning over code	–	–	–	✓	○	✓
Cyclomatic / structural metrics	○	–	○	–	–	✓
Dynamic / sandbox execution	–	–	–	–	–	✓
Unified composite risk score	–	–	–	–	–	✓
Developer-facing GUI with in-context triage	○	○	○	○	–	✓

(ii) Deeper static analysis: replace regex rules with an AST/code-property-graph layer [5] to add inter-procedural taint tracking. *(iii) Model scaling and routing*: support larger local models where hardware permits, and route only ambiguous files to the model to control latency. *(iv) Workflow integration*: a headless mode and a pre-commit / CI gate driven by a *Project Health Index* threshold, plus a controlled user study of triage speed against the usability criteria of [9], [10].

VII. CONCLUSION

We presented CODESENTINEL, a local-first desktop platform that unifies pattern-based static analysis, an on-device Llama 3.2 semantic review, containerised dynamic execution, and cyclomatic-complexity metrics into one privacy-preserving audit workflow, and folds their output into a single transparent *Project Health Index*. A worked case study on a representative retrieval-augmented-generation service showed the platform surfacing twenty findings—including prompt injection, unsafe deserialisation, server-side request forgery, and SQL injection—and five structural hot-spots, computing a moderate composite risk score, and completing in under a minute with no code leaving the machine. The contribution is not any single analysis technique but their co-location behind a coherent developer interface with a hard no-egress boundary. The main open problem is empirical: a controlled evaluation on labelled, real-world code is needed to quantify detection quality and to calibrate the scoring model. CODESENTINEL is available as open source [33].

DECLARATIONS

a) Availability of data and materials: The CODESENTINEL source code, the twelve interface figures used in this paper, and the definition of the case-study project are publicly available at https://github.com/ali-Hamza817/Code-Sentinel. No third-party or personal data were collected or processed.

b) Competing interests: The author declares no competing interests.

c) Funding: This work received no external funding.

d) Author contributions: A. Hamza designed and implemented the platform, designed and ran the case study, and wrote the manuscript.

e) Use of generative AI: A general-purpose AI assistant was used to help draft prose, produce the LaTeX and TikZ figure sources, and refine the wording of this manuscript. All technical claims, the architecture, the scoring model, the case-study construction, and the reported measurements were specified and verified by the author, who takes full responsibility for the content.

f) Acknowledgements: The author thanks the maintainers of the open-source projects on which CODESENTINEL depends, including Electron, Ollama, and the Llama model family.

g) Reproducibility note: Bibliographic details (page ranges and DOIs) should be re-verified against the publisher of record prior to submission; the accompanying `references.bib` carries the same note.

REFERENCES

[1] OWASP Foundation, "OWASP Top 10 – 2021," https://owasp.org/Top10/, 2021, accessed: Feb. 2026.
[2] The MITRE Corporation, "Common Weakness Enumeration (CWE)," https://cwe.mitre.org/, 2024, accessed: Feb. 2026.
[3] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76–79, 2004.
[4] N. Ayewah, W. Pugh, D. Hovemeyer, J. D. Morgenthaler, and J. Penix, "Using static analysis to find bugs," *IEEE Software*, vol. 25, no. 5, pp. 22–29, 2008.
[5] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2014, pp. 590–604.
[6] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
[7] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2023, pp. 2339–2356.
[8] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, "How secure is code generated by ChatGPT?" in *IEEE Int. Conf. on Systems, Man, and Cybernetics (SMC)*, 2023, pp. 2445–2451.
[9] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proc. 35th Int. Conf. on Software Engineering (ICSE)*, 2013, pp. 672–681.
[10] M. Nachtigall, M. Schlichtig, and E. Bodden, "A large-scale study of usability criteria addressed by static analysis tools," in *Proc. 31st ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, 2022, pp. 532–543.
[11] Ollama, "Ollama: Get up and running with large language models locally," https://ollama.com/, 2024, accessed: Feb. 2026.
[12] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[13] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.
[14] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," https://genai.owasp.org/llm-top-10/, 2025, accessed: Feb. 2026.
[15] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.
[16] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and D. Engler, "A few billion lines of code later: Using static analysis to find bugs in the real world," *Communications of the ACM*, vol. 53, no. 2, pp. 66–75, 2010.
[17] D. Distefano, M. Fähndrich, F. Logozzo, and P. W. O'Hearn, "Scaling static analyses at Facebook," *Communications of the ACM*, vol. 62, no. 8, pp. 62–70, 2019.
[18] P. Avgustinov, O. de Moor, M. P. Jones, and M. Schäfer, "QL: Object-oriented queries on relational data," in *30th European Conf. on Object-Oriented Programming (ECOOP)*, 2016, pp. 2:1–2:25.
[19] Semgrep, Inc., "Semgrep: Lightweight static analysis for many languages," https://semgrep.dev/, 2024, accessed: Feb. 2026.
[20] Python Code Quality Authority, "Bandit: A security linter for Python source code," https://bandit.readthedocs.io/, 2024, accessed: Feb. 2026.
[21] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
[22] D. Coleman, D. Ash, B. Lowther, and P. Oman, "Using metrics to evaluate software system maintainability," *Computer*, vol. 27, no. 8, pp. 44–49, 1994.
[23] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2021.
[24] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2022, pp. 754–768.
[25] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
[26] M. Fu and C. Tantithamthavorn, "LineVul: A transformer-based line-level vulnerability prediction," in *Proc. 19th Int. Conf. on Mining Software Repositories (MSR)*, 2022, pp. 608–620.
[27] B. Steenhoek, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *Proc. 45th Int. Conf. on Software Engineering (ICSE)*, 2023, pp. 2237–2248.
[28] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *Proc. 45th Int. Conf. on Software Engineering (ICSE)*, 2023, pp. 121–133.
[29] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," *arXiv preprint arXiv:2211.09527*, 2022.
[30] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
[31] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
[32] OpenJS Foundation, "Electron: Build cross-platform desktop apps with JavaScript, HTML, and CSS," https://www.electronjs.org/, 2024, accessed: Feb. 2026.
[33] A. Hamza, "CodeSentinel – source repository," https://github.com/ali-Hamza817/Code-Sentinel, 2026, accessed: Feb. 2026.